

Report Test run

Name

Release Testing WEGO (TR-2)



Report created

3/14/2026, 11:18 AM

Adrian Alexander



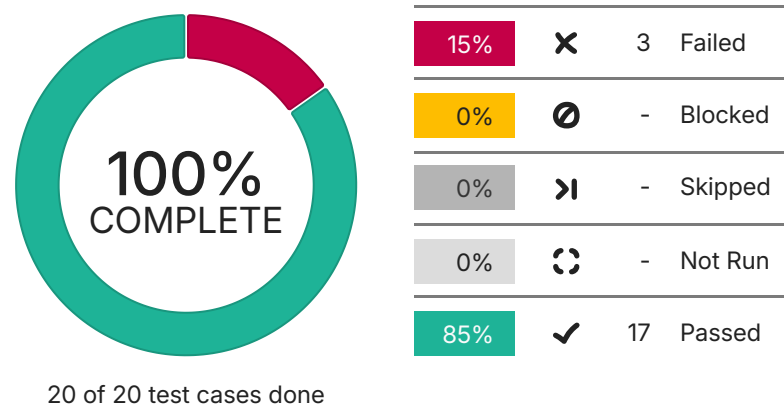
Test run created

2/22/2026, 5:31 PM

Adrian Alexander

📄

Test run status: Open



Test cases having no linked requirements (20)

ID	TITLE
✖ TC-17	Cross-Platform Performance Baseline (Lighthouse)
✖ TC-20	Verify UI Update After Successful Login
✖ TC-21	Lack of Server-Side Input Sanitization for 'Address' parameter
✔ TC-2	Hidden Sort button
✔ TC-3	XSS Security
✔ TC-4	Authentication Token Should Not Be Stored in document.cookie
✔ TC-5	Manual Log in
✔ TC-6	XSS Validation on Search UI
✔ TC-7	XSS Injection in Search Query Parameter – URL Reflected Content
✔ TC-8	Reset Password Email – Verify no XSS Injection is possible
✔ TC-9	Validate API response time, content-type, and JSON structure for GET endpoint
✔ TC-10	Spike Test – High-Frequency GET Request Execution (150 iterations, 1 ms delay)
✔ TC-11	Spike Test – Sudden Load Increase from 20 to 200 VUs over 2 Minutes (API Endpoint)
✔ TC-12	Peak Load Test – Maintain 200 Virtual Users for 2 Minutes (Base Load 40)
✔ TC-13	TEST CASE – Reflected XSS – Double URL Decoding Protection
✔ TC-14	HTTP Security Headers Validation
✔ TC-15	IDOR Protection on Profile Update API
✔ TC-16	Magic Link Security & Cross-Location Validation
✔ TC-18	Automated Navigation & Link Integrity Audit (Broken Links)

ID	TITLE
 TC-19	Endurance (Soak) Test

Test case details (20)

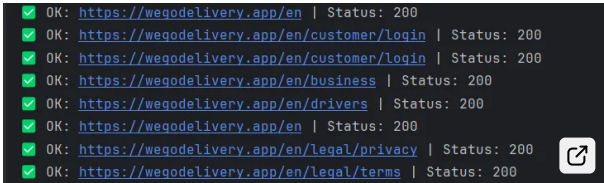
ID	TITLE	TESTED BY	TESTED AT
	 APitests		
 TC-9	 Validate API response time, content-type, and JSON structure for GET endpoint	Adrian Alexander	2/25/2026, 10:27 AM

TEST CASE DETAILS	
PRIORITY	TYPE
High	Performance
PRECONDITION	
Postman installed	
Collection-level tests configured:	
Response time < 2000ms	
Content-Type contains application/json	
Response body must be valid JSON	
STEPS	EXPECTED RESULTS
Step 1 — Send GET request to endpoint	
Example: GET	Validation Expected
https://wegodelivery.app/api/subscription-plans 	HTTP Status 200 OK
	Response Time < 2000ms
Step 2 — Allow Postman to execute collection-level test script	Content-Type application/json
	JSON Body valid JSON, parseable
Script used:	
// Collection-level tests: applied after every request	
pm.test('Response time is under 2000ms', function () {	
pm.expect(pm.response.responseTime).to.be.below(2000);	
});	
pm.test('Content-Type header contains application/json', function () {	
pm.response.to.have.header('Content-Type');	
pm.expect(pm.response.headers.get('Content-Type')).to.include('application/json');	
});	
pm.test('Response body is valid JSON', function () {	
pm.response.to.be.json;	
pm.response.json();	
});	
Step 3 — Observe response details	
Response time	

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
Content-Type header JSON body			
Step 4 — Verify HTTP status code Expected: 200 OK			

 FunctionalTests.

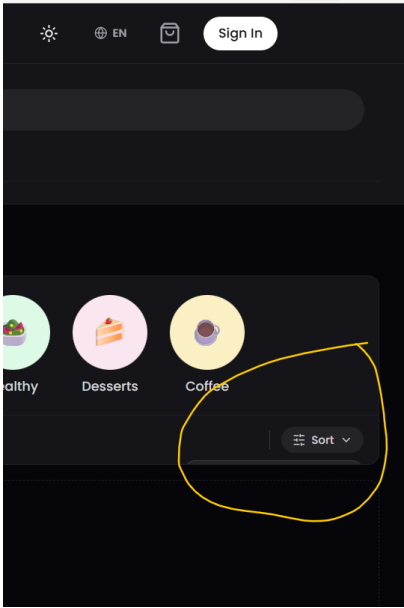
	TC-18	 Automated Navigation & Link Integrity Audit (Broken Links)	Adrian Alexander	3/8/2026, 6:12 PM
--	-----------------------	--	------------------	-------------------

TEST CASE DETAILS	
PRIORITY	TYPE
High	Functional
PRECONDITION	
The testing framework (IntelliJ IDEA) is correctly configured and all dependencies are loaded.	
The LinkUtils class is present in the utils package/folder.	
The @BrokenLinks Cucumber scenario is defined and available in the features folder.	
Active internet connection is required to validate external/live endpoints.	
STEPS	EXPECTED RESULTS
1. Open IntelliJ IDEA and navigate to the BrokenLinks.feature file.	HTTP Status Codes: All extracted links must return a 200 OK status or a valid redirect code (301/302).
2. Execute the scenario using the @BrokenLinks tag (or run the RunTest class).	No 4xx/5xx Errors: No links should return "Client Errors" (e.g., 404) or "Server Errors" (e.g., 500).
3. Monitor the Console Output for real-time verification logs.	Specific Endpoint Validation: The "Help" navigation link must be functional and reachable.
4. Identify and document any links flagged with the  BROKEN prefix.	

 ManualTests

	TC-2	 Hidden Sort button	Adrian Alexander	2/24/2026, 12:46 PM
--	----------------------	--	------------------	---------------------

TEST CASE DETAILS	
PRIORITY	TYPE
High	Functional
PRECONDITION	
User must be on page https://wegodelivery.app/en/search	
STEPS	
1  Given that user in on search restaurant page: https://wegodelivery.app/en/search	 Expected result:

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
		The page loads correctly and the Sort button is visible on the right side.	
2	🔗 When user clicks on Sort button to the right side	✅ Expected result: The Sort dropdown should open.	
3	🔗 Then the drop down menu should be visible	✅ The dropdown is not hidden, not cut off, and all sorting options are readable.	
ATTACHMENTS			
📎 Screenshot 2026-02-22 173104.png		Adrian Alexander	2/22/2026, 5:31 PM
			
RESULT COMMENTS & ATTACHMENTS			
Claudiu Sfetan 3 days ago Fix committed — 4a32bcc on main			
Root cause: The filter container wrapping the sort dropdown had overflow-hidden CSS class. This creates a clipping boundary that hides absolutely-positioned children (the dropdown menu) regardless of their z-index value.			
💬	Fix: Removed overflow-hidden from the filter container in search-content.tsx:530. The sort dropdown (with z-[100]) now renders above the container as intended.	Adrian Alexander	2/27/2026, 4:06 PM
File changed: apps/web/src/app/[locale]/(customer)/search/_components/search-content.tsx			

ID	TITLE	TESTED BY	TESTED AT
 TC-20	 Verify UI Update After Successful Login	Adrian Alexander	3/11/2026, 12:40 PM

TEST CASE DETAILS	
PRIORITY	TYPE
Medium	Functional
PRECONDITION	
User has a valid registered account.	
User is on the homepage (https://wegodelivery.app/en) and is currently logged out	
STEPS	EXPECTED RESULTS
1. Click on the "Sign in" button.	The message "Already have an account? Sign in" must be hidden.
2. Use valid email and password.	
3. After being redirected to the homepage, observe the section below the search bar.	The UI should display the user's profile icon or a "Logout" option instead of the login CTA.
4. Manually refresh the page (F5) to check if the UI state persists.	The authentication state must persist even after a page refresh.

 TC-5	 Manual Log in	Adrian Alexander	2/24/2026, 12:47 PM
---	---	------------------	---------------------

TEST CASE DETAILS	
PRIORITY	TYPE
High	Accessibility
PRECONDITION	
User must be on login page: https://wegodelivery.app/en/customer/login 	
STEPS	EXPECTED RESULTS
Add a fake email address	A verification email should be sent only to valid Email addresses and denied for invalid ones
Click Continue	

RESULT COMMENTS & ATTACHMENTS	AUTHOR	COMMENT DATE
 Claudiu Sfetan 3 days ago Fix Deployed — 0b626ae on main Issue 1: Fake emails accepted Root cause: The email validation only checked RFC format (Zod .email()), with no domain-level filtering. Any syntactically valid email like test@mailinator.com would pass and trigger a Supabase OTP magic link send. Fix: Added a disposable email domain blocklist (~100 known providers) to the emailSchema in customer-auth-schemas.ts. The check: Runs after email normalization (trim + lowercase) Uses efficient Set lookup (O(1)) Applies to both customer login and signup flows (shared schema)	Adrian Alexander	2/27/2026, 4:08 PM

ID	TITLE	TESTED BY	TESTED AT
RESULT COMMENTS & ATTACHMENTS		AUTHOR	COMMENT DATE
Shows error: "Temporary or disposable email addresses are not allowed" Blocks: mailinator.com, yopmail.com, guerrillamail.com, temp-mail.org, and ~96 more Tests: 5 new test cases added, all passing (38 total). Issue 2: Real emails — verification sent but never received This is a Supabase SMTP delivery issue outside our application code. Supabase Auth handles magic link email sending via their own SMTP infrastructure. Possible causes: Emails landing in spam folders Supabase email rate limits Email provider delivery delays This should be investigated in the Supabase dashboard under Auth > Email Templates and Logs. Files Changed apps/web/src/lib/auth/disposable-email-domains.ts — new blocklist (100 domains) apps/web/src/lib/auth/customer-auth-schemas.ts — email schema with disposable check apps/web/src/lib/auth/customer-auth-schemas.test.ts — 5 new test cases			



Performance



TC-10



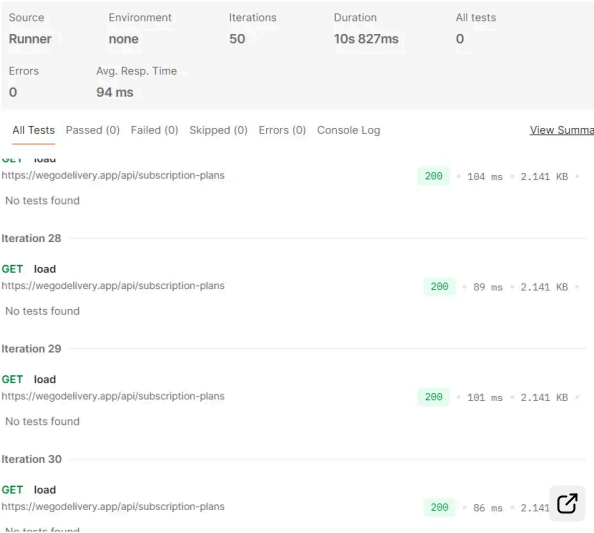
Spike Test – High-Frequency GET Request Execution (150 iterations, 1 ms delay)

Adrian Alexander

2/25/2026, 12:53 PM

TEST CASE DETAILS

PRIORITY	TYPE
High	Performance
PRECONDITION	
Postman installed	
Target request saved inside a Collection	
Endpoint accessible	
Authentication (if required) is valid	
Stable internet connection	
No payload (GET request)	
Runner settings:	
Iterations: 150	

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
Delay: 1 ms			
STEPS		EXPECTED RESULTS	
Open Postman and create a new Collection		All 150 requests return HTTP 200 OK.	
Create new get request with endpoint: https://wegodelivery.app/api/subscription-plans		Response Time remains within acceptable limits (< 150–200 ms).	
Click Run Collection.			
Set Iterations = 50.		No signs of throttling (429).	
Set Delay = 2 ms.			
Start the collection run.		No server-side errors (500/503).	
Observe the results panel for each iteration:			
Response Time		No session invalidation or unexpected authentication behavior	
Status Code			
Body consistency			
Verify if any errors appear (429, 500, 503, timeouts).			
Note any fluctuations in performance or instability.			

✓

TC-11

Spike Test – Sudden Load Increase from 20 to 200 VUs over 2 Minutes (API Endpoint)

Adrian Alexander

2/25/2026, 4:12 PM

TEST CASE DETAILS	
PRIORITY	TYPE
High	Performance
PRECONDITION	
Postman Desktop version with Performance Testing enabled	
Target API endpoint available	
Network stable	
The following endpoint is added and saved in a Postman Collection: https://wegodelivery.app/api/subscription-plans	
STEPS	EXPECTED RESULTS
Open Postman.	System should handle the sudden traffic increase without failure.
Navigate to the Collection containing this request:	Error % must remain < 1%.
GET https://wegodelivery.app/api/subscription-plans	No 500/503 server errors.
Click Run Collection.	No 429 Too Many Requests (unless rate limiting intentionally applied).
Select Performance mode.	

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
Choose Spike Testing. Enter the spike parameters: Start VUs: 20 spike duration: 2 minutes Spike to: 200 virtual users Start the Spike Test. Monitor real-time metrics: Requests per second Average latency P90 / P95 / P99 latency Error rate Failures		Average response time should remain under 200–250 ms, even at peak. P99 should ideally remain under 500 ms. SummaryErrorsTests Total requests sent ⓘRequests/second ⓘAvg. response time ⓘP90 ⓘP95 ⓘP99 ⓘError % ⓘFailure % ⓘ 55,910464.8645 ms62 ms69 ms93 ms0.000.00 Filter by requests ▾Avg. response ▾ % 100806040200 169 ms1,600 req/s1501,4001,2001,0008006004002000 Requests/secondAvg. responseError %Virtual users Performance details for total duration Request Total requestsRequests/sError %Failure %R ... GET load55,910461.230.000.00	



IC-17

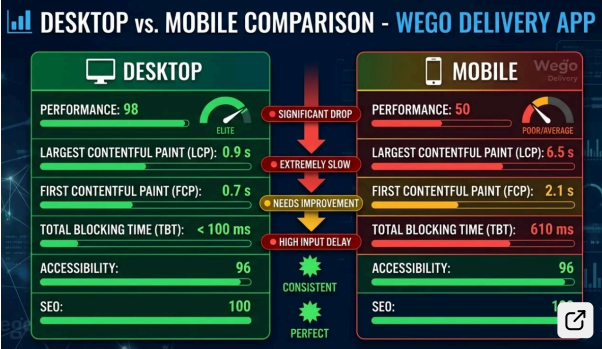


Cross-Platform Performance Baseline
(Lighthouse)

Adrian Alexander

3/8/2026, 11:25 AM

TEST CASE DETAILS	
PRIORITY	TYPE
High	Performance
PRECONDITION	
Browser: Use a Chromium-based browser (Google Chrome or Microsoft Edge). Environment: Open a new Incognito/Private window to ensure browser extensions or cached data do not interfere with the performance score. Network: Ensure a stable internet connection (preferably a high-speed fiber connection for the Desktop baseline). Hardware: Close all other heavy applications (like IntelliJ, Slack, or Docker) on the local machine to avoid CPU throttling during the audit. URL: Ensure the Wego production URL is accessible and not behind a maintenance page or a CAPTCHA.	
STEPS	EXPECTED RESULTS
1. Open Browser in Incognito mode.	Desktop Performance: Score should be > 90, with LCP < 1.2s and TBT < 100ms.
2. Navigate to the Wego homepage.	Mobile Performance: Score should be > 80 (or at least within a 15% margin of the Desktop score).
3. Open DevTools (F12) and select the Lighthouse tab.	Core Web Vitals: Largest Contentful Paint (LCP) must be under 2.5s for both platforms to meet Google's "Good" threshold.
4. Set "Mode" to Navigation, "Device" to Desktop, and select all categories.	Consistency: Accessibility, Best Practices, and SEO scores should remain > 90 across both devices.
5. Click "Analyze page load" and wait for the report.	No Critical Regressions: The mobile LCP should not exceed 3s (currently it is at 6.5s)
6. Record the scores for Performance, LCP (Largest Contentful Paint), and TBT (Total Blocking Time).	
7. Clear the report and change the "Device" setting to Mobile.	
8. Click "Analyze page load" again and wait for the report.	

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
9. Compare the Mobile metrics against the Desktop baseline.			

✓	TC-19	📅 Endurance (Soak) Test	Adrian Alexander	3/9/2026, 3:50 PM
---	-------	-------------------------	------------------	-------------------

TEST CASE DETAILS	
PRIORITY	TYPE
High	Performance
PRECONDITION	
Application must be online and responsive	
User must have an account an be logged in.	
Postman installed	
Stable connection to internet	
STEPS	EXPECTED RESULTS
1. Open Postman and create new Collection	Status Code Consistency: All requests (GET, POST, and PATCH) must return a 200 OK status code consistently throughout the 30-minute duration.
2. Create 1 request GET with URL: https://wegodelivery.app/api/subscription-plans	
3. Create 2nd request POST with URL: https://wegodelivery.app/api/places/autocomplete	Performance SLA Compliance: The Average Response Time must remain stable and below the 500ms threshold under the constant load of 20 Virtual Users.
4. Add Body RAW JSON: {"input": "{{randomCity}}"} (to prevent server-side caching).	Zero Error Rate: The total error rate must be 0% (or < 1%), indicating that no requests are dropped or rejected by the server during the soak test.
5. Add Header Cookie using a Collection Variable (e.g., {{auth_cookie}}).	No Performance Degradation: The response time graph should remain relatively flat, with no significant upward trends, confirming there are no memory leaks or resource exhaustion issues on the server.
6. Add 3rd request PATCH with URL: https://wegodelivery.app/api/profile	Concurrent Data Processing: The server must correctly handle simultaneous PATCH requests to the /api/profile endpoint without data corruption or service interruption.
7. Add BODY RAW Json : {"full_name": "{{randomFullName}}"} 8. Add Header Cookie with the same Value as in step 5	
9. Save all request before stating the test	
10. Right click on the collection and select RUN	
11. Click on Performance tab and use FIXED profile, VU=20, and test duration 30 minutes	
12. Configure Pass/Fail Criteria: Set Average Response Time < 500ms and Error Rate < 1%.	
13. Click RUN	

ID

TITLE

TESTED BY

TESTED AT

TEST CASE DETAILS

api load (171) Pass

api load - 20 VUs - Mar 08, 2026 12:24:53 (30 mins) - Fixed profile

SummaryErrorsTests

Total requests sent	Requests/second	Avg. response time	P90	P95	P99	Error %	Failure %
274,936	152.71	112 ms	136 ms	154 ms	212 ms	0.00	0.00

Filter by requests

Avg. response

% 100

80

60

40

20

0

12:25

12:28

12:31

12:34

12:37

12:40

12:43

12:46

12:49

12:52

Requests/second

Avg. response

Error %

Virtual users

536 ms

200 req/s

500

400

300

200

100

0

Performance details for total duration

Request	requests	Requests/s	Error %	Failure %	Resp. time (Avg. ms)	Min (ms)	Max (ms)	P90	P95	...
GET /load	91,651	50.89	0.00	0.00	104	64	1,553	126	142	
POST SearchAPI	91,645	50.89	0.00	0.00	118	53	75,587	140	151	
WATCH APIProfile	91,640	50.89	0.00	0.00	115	67	76,178	139	15	

✓	TC-12	Peak Load Test – Maintain 200 Virtual Users for 2 Minutes (Base Load 40)	Adrian Alexander	2/25/2026, 4:36 PM
---	-------	---	------------------	--------------------

TEST CASE DETAILS													
PRIORITY High PRECONDITION Postman Desktop Performance Testing enabled Endpoint added to Collection: GET https://wegodelivery.app/api/subscription-plans Stable network	TYPE Performance												
STEPS Open Postman. Select the Collection containing the request: https://wegodelivery.app/api/subscription-plans Click Run Collection. Select Performance Mode. Choose Peak Load Test profile. Configure the following settings: Peak Load: 200 VUs Duration: 2 minutes Start the test run. Monitor live metrics: Requests/sec Average response Latency percentiles (P90, P95, P99) Error rate	EXPECTED RESULTS System should maintain stable performance at 200 VUs. No backend errors (500 / 503). No rate limiting (429). Error rate below 1%. Average response under 200 ms. P99 under 500 ms. SummaryErrorsTests Total requests sentRequests/secondAvg. response timeP90P95P99Error %Failure % 62,900520.1755 ms72 ms86 ms167 ms0.000.00 Filter by requestsAvg. response % 100 80 60 40 20 0 259 ms1,600 req/s 1,400 1,200 1,000 800 600 400 200 0 Requests/secondAvg. responseError %Virtual users Performance details for total duration <table><tr><th>#</th><th>Request</th><th>Total requests</th><th>Requests/s</th><th>Error %</th><th>Fi</th></tr><tr><td>1</td><td>GET load</td><td>63,000</td><td>516.18</td><td>0.00</td><td></td></tr></table>	#	Request	Total requests	Requests/s	Error %	Fi	1	GET load	63,000	516.18	0.00	
#	Request	Total requests	Requests/s	Error %	Fi								
1	GET load	63,000	516.18	0.00									

ID	TITLE	TESTED BY	TESTED AT
	 SecurityTests		
	TC-3  XSS Security	Adrian Alexander	2/24/2026, 3:52 PM
TEST CASE DETAILS			
PRIORITY		TYPE	
Critical		Security	
PRECONDITION			
User must be logged in and on the Profile / Account Settings page: https://wegodelivery.app/en/account/personal-info 			
STEPS			
1	 Given the user is on the Profile page where the Name field can be edited	 Expected Result: The Name input field is visible, active, and ready for user input.	
2	 When the user enters the following text into the Name field: <code></code>	 Expected Result: The input is temporarily accepted into the field without immediate script execution in the browser.	
3	 And the user clicks the Save / Update Profile button	 Expected Result: The request is submitted to the backend. The system processes the update without triggering any JavaScript execution.	
4	 Then the script should not execute and no alert should appear	 Expected Result: No JavaScript popup is triggered The Name field value is sanitized The saved name appears cleaned (e.g., <code><script></code> removed or escaped) No XSS payload is stored or executed	
RESULT COMMENTS & ATTACHMENTS		AUTHOR	COMMENT DATE
	Claudiu Sfetan	Adrian Alexander	2/27/2026, 4:09 PM
3 days ago Fix Deployed — 63852ad on main Investigation React's JSX auto-escaping already prevents XSS when rendering — dangerouslySetInnerHTML is not used anywhere in the codebase. A payload like <code><script>alert('XSS')</script></code> entered in the profile name field would display as literal text, not execute. However, storing unsanitized HTML in the database is still a defense-in-depth concern (future code could introduce unsafe			

ID	TITLE	TESTED BY	TESTED AT
RESULT	COMMENTS & ATTACHMENTS	AUTHOR	COMMENT DATE
	rendering patterns). Changes Made Extracted sanitizeTextInput() to @wego/utls — shared function that strips HTML tags (<[^>]*>), unclosed < characters, and control characters including DEL (\x7F) Applied .transform(sanitizeTextInput).pipe(z.string().min(N)) to all user-facing text fields: Restaurant name, description Menu category name, description Menu item name, description Modifier group name, description Modifier name Order notes, reject/cancel reasons Special instructions (cart, checkout, order items, orders) Delivery instructions Restaurant and driver reviews Profile fullName Post-sanitization validation — .pipe() re-validates min-length after HTML stripping, so a payload like <script> (8 chars, passes min(1)) gets stripped to "" and correctly rejected Files Changed packages/utls/src/index.ts — new shared sanitizeTextInput export packages/utls/src/restaurant-schemas.ts — sanitization on all text fields packages/utls/src/customer-schemas.ts — sanitization on all text fields apps/web/src/app/api/profile/route.ts — uses shared import instead of local copy		

ID	TITLE	TESTED BY	TESTED AT
 TC-21	 Lack of Server-Side Input Sanitization for 'Address' parameter	Adrian Alexander	3/12/2026, 12:55 PM
TEST CASE DETAILS PRIORITY High TYPE Security PRECONDITION Application must be available Burp Suite community edition installed and ready STEPS 1. Open Burp Suite 2. Click on Proxy and make sure Intercept is OFF 3. Open Browser from Burp and go to WeGo Homepage: https://wegodelivery.app/en 4. Type any city in the "enter delivery address" parameter, but don't Hit Enter 5. In Burp turn ON Intercept 6. Return to the browser and hit enter 7. Return to Burp and click on the request that shows the city you entered 8. In the Inspector section change the value of your city with <code></code> 9. Click Forward 10. Check the history section and the browser 11. Analyze the server response in Burps history EXPECTED RESULTS 1. Expected Result: The server should sanitize or reject special characters (<, >, ").			

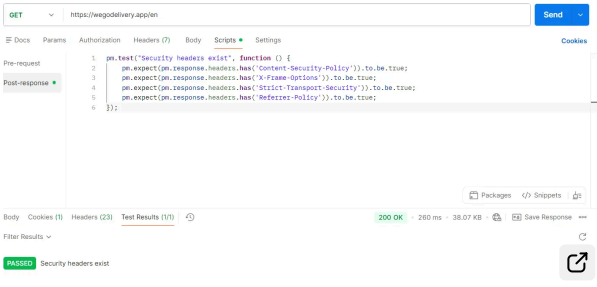
 TC-16	 Magic Link Security & Cross-Location Validation	Adrian Alexander	3/8/2026, 12:06 PM
TEST CASE DETAILS PRIORITY High TYPE Security PRECONDITION The application is online. User has a fresh email available or a business email Access to a VPN service (for simulating different geographic locations). Access to the email inbox associated with the account. STEPS 1. Set VPN to any Country and navigate to the Wego SignIn page. 2. Enter the email address and click the "Continue" button. 3. Close the browser completely and change the VPN location to a different location. 4. Open the email and click the Magic Link received. 5. Perform a manual Logout from the application. 6. Return to the email and click the same Magic Link a second time. EXPECTED RESULTS The system permits "IP Roaming"; the user is successfully logged into the Dashboard in Step 4 despite the geographic location change. The session is terminated correctly upon Logout. The Magic Link is invalidated after the first use, regardless of the location; Step 6 results in an error and access is denied, being redirected to Sign in page to a fresh start.			

ID	TITLE	TESTED BY	TESTED AT
 TC-15	 IDOR Protection on Profile Update API	Adrian Alexander	3/6/2026, 9:28 PM

TEST CASE DETAILS	
PRIORITY High	TYPE Security
PRECONDITION Application available Two valid user accounts (User A and User B). The UUID (ID) of User B is known. Postman installed.	
STEPS Identify Target ID (User B - The Victim) Login to the application as User B. Open Console (F12) and type: document.cookie. Locate the auth-token (the Base64 encoded JSON) and extract the access_token string (starts with eyJ...). Go to jwt.io and paste the token into the "Encoded" section. In the Payload section, locate and copy the id (or sub) field. This is the victim's UUID. II. Prepare the Attack (User A - The Attacker) Login to the application as User A in the browser. Open Network Tab (F12) and filter by Fetch/XHR. Perform a profile update (e.g., change full name) and click Save. Locate the PATCH request to /api/profile. Right-click the request → Copy → Copy as cURL (bash). III. Execute the IDOR Test (Postman) In Postman, click Import → Raw Text and paste the cURL. In the Body tab, modify the JSON payload: <pre>{ "id": "user ID", "full_name": "Hacked by IMe" }</pre> Click Send.	EXPECTED RESULTS Check Status Code: Should be 403 Forbidden (ideal) or 200 OK (if the server defaults to User A's session). Verify User B: Log in as User B and check the profile. Name must remain unchanged. Verify User A: If status was 200 OK, check User A's profile. If User A's name changed instead of User B's, the system is safe from IDOR.

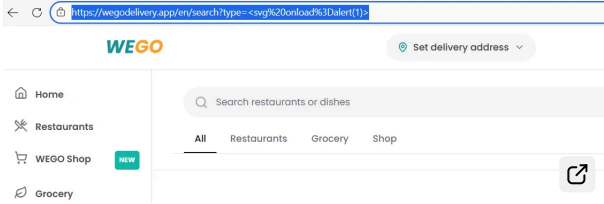
 TC-14	 HTTP Security Headers Validation	Adrian Alexander	2/26/2026, 1:38 PM
--	--	------------------	--------------------

TEST CASE DETAILS	
PRIORITY High	TYPE Security
PRECONDITION Postman installed	

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
Internet connection available User has permission to perform security assessment Target domain reachable online Postman request configured with method: GET No browser extensions or proxies modifying headers			
STEPS		EXPECTED RESULTS	
Open Postman. Create a GET request to: https://wegodelivery.app/en Navigate to the Tests tab in Postman. Paste the following script into the Tests tab: <pre>pm.test("Security headers exist", function () { pm.expect(pm.response.headers.has('Content-Security-Policy')).to.be.true; pm.expect(pm.response.headers.has('X-Frame-Options')).to.be.true; pm.expect(pm.response.headers.has('Strict-Transport-Security')).to.be.true; pm.expect(pm.response.headers.has('Referrer-Policy')).to.be.true; });</pre> Click Send. After the response is received, scroll down to the Test Results section. Verify: If the script reports PASS, all required security headers exist. If it reports FAIL, review which header is missing. Additionally, open the Headers → Response Headers tab and manually validate each header value.		Postman script returns PASS in the Test Results section. All required security headers are present in the response: Content-Security-Policy X-Frame-Options Strict-Transport-Security Referrer-Policy No critical security header is missing. No insecure wildcard (*) is used.	
			

	TC-13	TEST CASE – Reflected XSS – Double URL Decoding Protection	Adrian Alexander	2/25/2026, 7:11 PM
--	-----------------------	--	------------------	--------------------

TEST CASE DETAILS	
PRIORITY	TYPE
High	Security
PRECONDITION	
The application wegodelivery.app is accessible and operational.	
No authentication is required for accessing the /en/search endpoint.	
A modern web browser is available (Google Chrome recommended).	
No browser extensions are active that could alter or block JavaScript execution (e.g., NoScript, strict ad-blockers, script blockers).	
A stable internet connection is available.	
STEPS	EXPECTED RESULTS
Open a modern web browser (Google Chrome recommended).	The payload remains URL-encoded in the browser after the page loads.

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
In the address bar, enter the following URL: https://wegodelivery.app/en/search?type=%3Csvg%20onload%3Dalert(1)%3E Press Enter to load the page. After the page loads, observe the URL displayed in the browser's address bar. Copy the final URL and verify whether the payload remains URL-encoded. Check the rendered page content to determine whether the payload appears as decoded HTML (e.g., <svg onload=alert(1)>). Verify that no SVG element is injected into the DOM. Confirm that no JavaScript execution occurs (no alert(1) pop-up or other script execution). Ensure that the payload, if displayed, is properly escaped (e.g., &lt;svg onload=alert(1)&gt;).		The application does not perform double URL decoding. The payload is not interpreted as executable HTML or JavaScript. No SVG element is rendered in the DOM. No JavaScript execution occurs (no alert dialog appears). If the payload is reflected in the page, it is properly escaped and displayed as plain text. The endpoint is not vulnerable to reflected XSS via double URL decoding. 	

✓	TC-7	XSS Injection in Search Query Parameter – URL Reflected Content	Adrian Alexander	2/24/2026, 4:30 PM
---	------	---	------------------	--------------------

TEST CASE DETAILS	
PRIORITY	TYPE
-	-
PRECONDITION	
User is on https://wegodelivery.app	
Browser DevTools open → Elements and Network	
No browser extensions interfering (adblock, scripts blockers)	
STEPS	EXPECTED RESULTS
1. Navigate to URL with XSS payload	✖ Expected Result
Open the following URL in the browser:	The application correctly escapes or encodes user-supplied input.
https://wegodelivery.app/en?search=<svg/onload=alert(1)>	The XSS payload is NOT executed.
2. Observe UI behavior	The payload stored in the iframe is encoded, not interpreted.
Page loads normally	No reflected XSS is present.
No alert pops up	
No JavaScript execution	
3. Open DevTools → Elements	
Search for your payload:	
Press CTRL+F	
Search for:	
<svg	
alert	
<script	
Expected: Nothing appears in DOM as executable content.	

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
4. Check if the payload was escaped Locate the value reflected in the Stripe iframe (optional but useful): You should see something similar to: <pre>referrer=...%253Csvg%252Fonload%253Dalert(1)%253E</pre> → It should be double-encoded (%253C ≠ <) 5. Confirm that no XSS is executed No popup No DOM injection No script execution Payload is treated as text, not HTML			

TC-8

Reset Password Email – Verify no XSS Injection is possible

Adrian Alexander

2/24/2026, 5:32 PM

TEST CASE DETAILS	
PRIORITY	TYPE
Medium	Security
PRECONDITION	
User has an active WEGO account.	
User is logged into the application OR has access to the “Forgot Password” page.	
Email inbox accessible (Gmail).	
XSS payloads already tested in profile fields (name, address, etc.).	
STEPS	EXPECTED RESULTS
Phase 1 — Inject Payload into Profile	✔ The payload does NOT appear anywhere in the email.
Log in to the WEGO website.	
Navigate to: Account → Personal Information.	
In the Name field, input the XSS payload:	✔ The email template remains static and unmodified.
\x3Csvg/onload=alert(1)\x3E	
Click Save Changes.	✔ No script executes when the email is opened.
Confirm that the payload is stored in the UI (it appears as text, escaped).	✔ No malformed HTML in the email.
Phase 2 — Trigger Password Reset	✔ Personal name (where payload was injected) is not included in the email template.
Log out OR open Forgot Password page directly.	
Navigate to:	
https://wegodelivery.app/en/login	✔ Reset URL contains only Supabase-generated:
Click “Forgot your password?”.	token=
Enter the email of the account that contains the stored payload in its profile.	
Submit the form.	redirect_to=

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
Phase 3 — Inspect Received Email		✔ No stored XSS reaches the email system.	
Open Gmail. Find the email titled "Reset Your Password" from noreply@wegodelivery.app .Open the email normally (HTML view). Click More → Show Original to view the raw email. Search the entire raw email for the payload text: \x3Csvg/onload=alert(1)\x3E <svg/onload=alert(1)> "><script>alert(1)</script> Confirm whether the payload appears in any of the following: Email subject Email HTML body Email plaintext body Reset password button/link Any metadata or hidden fields			

ID	TITLE	TESTED BY	TESTED AT
TC-6	XSS Validation on Search UI	Adrian Alexander	2/23/2026, 10:30 PM
TEST CASE DETAILS PRIORITY High TYPE Security PRECONDITION User is logged into the WEGO web platform Access to Search functionality Dev Tools → Elements & Network tab available Browser CSP is active STEPS 1 Navigate to:https://wegodelivery.app/en/search 2 Type or paste in the search bar:TEST Expected Result (Step 2) UI displays the text exactly as typed, escaped as: &lt;b>TEST&lt;/b> The text is NOT rendered as bold HTML No formatting changes appear. No DOM injection in Elements tab. No script execution.			
TC-4	Authentication Token Should Not Be Stored in document.cookie	Adrian Alexander	2/24/2026, 4:35 PM

TEST CASE DETAILS	
PRIORITY	TYPE
Critical	Security
PRECONDITION	
User is logged into the application.	
Browser DevTools Console is accessible.	
User is on any authenticated page of the application	
STEPS	EXPECTED RESULTS
Step 1 Open the browser DevTools (F12) and navigate to the Console tab.	No authentication tokens, JWTs, Bearer tokens, or other sensitive information are stored in document.cookie. Sensitive authentication data is NOT accessible from JavaScript.
Step 2 Execute the following command in the Console: document.cookie	Tokens are stored in an HttpOnly cookie or using secure storage mechanisms.
Step 3 Inspect the cookie string for sensitive data such as: token auth jwt	

ID	TITLE	TESTED BY	TESTED AT
TEST CASE DETAILS			
bearer			
Step 4			
Validate that authentication tokens are stored using secure methods (e.g., HttpOnly cookies or local/session storage).			
RESULT COMMENTS & ATTACHMENTS		AUTHOR	COMMENT DATE
 Claudiu Sfetan		Adrian Alexander	2/27/2026, 4:12 PM
3 days ago			
Investigation Result: By Design (Supabase SSR Architecture)			
Research Summary			
After researching Supabase's official documentation and community discussions, this is expected behavior for Supabase Auth with SSR — not a vulnerability in our implementation.			
Supabase official docs (How do I make the cookies HttpOnly?):			
"This is not necessary. Both the access token and refresh token are designed to be passed around to different components in your application. The browser-based side of your application needs access to the refresh token to properly maintain a browser session anyway."			
Supabase maintainer @hf (GitHub Discussion #12303):			
"HTTP-Only cookies are not an option unless you're making a very traditional web app."			
Why HttpOnly cookies would break our app			
Setting cookies to httpOnly would disable:			
onAuthStateChange() — client can't detect login/logout			
Client-side getUser() — all React Query hooks break			
Supabase Realtime subscriptions (order tracking, driver location)			
All Zustand store auth flows			
Mitigations already in place			
Our cookie-options.ts follows Supabase's recommended best practices:			
Defense			
Status			

ID	TITLE	TESTED BY	TESTED AT
RESULT COMMENTS & ATTACHMENTS		AUTHOR	COMMENT DATE
secure: true (HTTPS-only in production)			
Implemented			
sameSite: 'lax' (CSRF protection)			
Implemented			
maxAge: 7 days (reduced from Supabase's 400-day default)			
Implemented			
Content Security Policy (CSP) with strict allowlists			
Implemented			
CSP violation reporting endpoint			
Implemented			
Strict-Transport-Security (HSTS with preload)			
Implemented			
X-Frame-Options: DENY (clickjacking prevention)			
Implemented			
X-Content-Type-Options: nosniff			
Implemented			
Conclusion			
The attack scenario described ("if an attacker can access the browser through XSS") is mitigated by preventing XSS in the first place, which is handled by our CSP headers and addressed further in KAN-9 (Stored XSS fix). Making cookies HttpOnly is not feasible with Supabase's architecture and is explicitly not recommended by Supabase.			